

Production-Ready AI Brokerage Assistant

Complete Delivery Roadmap for Claude Code

Project Goal

Build a state-of-the-art multilingual brokerage AI assistant that:

- Supports English, Hindi and Tamil
 - Can be embedded into any brokerage website
 - Provides stock information
 - Explains company disclosures and filings
 - Helps users navigate the website
 - Explains brokerage services
 - Provides NALCO-focused disclosure intelligence
 - Supports future algo-trading education and workflows
 - Scales to 100+ concurrent users
 - Runs on Railway Pro
 - Uses Ollama Cloud (Gemma Model)
 - Uses Qdrant as the vector database
 - Uses FastAPI as the backend
 - Uses a modern React / Next.js frontend
-

Phase 0 – Research & Competitive Analysis

Before writing code, analyze:

Competitors

Brokerage Chatbots

- Zerodha
- Angel One
- Groww
- Upstox
- ICICI Direct

AI Chat Platforms

- Botpress
- ManyChat
- Intercom AI
- Drift AI
- Freshchat AI

Analyse

For each competitor:

User Experience

- Homepage experience
- Chat experience
- Mobile experience

AI Features

- Language support
- Search capability
- Document retrieval

- Stock information

Lead Generation

- Account opening assistance
- Human handoff
- Call booking

Identify Gaps

Questions:

- What are they missing?
 - What frustrates users?
 - Where can we differentiate?
-

Phase 1 – UX Design

Design before coding.

Deliverables

Website Wireframes

Pages:

- Home
- Stock Research
- NALCO Intelligence
- Algo Education
- Pricing
- Contact

Chatbot Widget

Features:

- Floating chatbot
- Full-screen chatbot
- Mobile responsive

Multilingual UI

Languages:

- English
 - Hindi
 - Tamil
-

Phase 2 – System Architecture

Deployment Platform

Railway Pro

Architecture:

Users

↓

Website
↓
Next.js Frontend
↓
FastAPI Backend
↓
Qdrant
↓
Ollama Cloud

- External Services:
-
- LLM Layer
- - Ollama Cloud (Gemma)
-
- Language Layer
- - Sarvam AI API
-
- Market Data Provider Layer

Architecture Principle

The application must not be tightly coupled to any single market data vendor.

All market data providers must implement a common interface.

Examples:

MarketDataProvider

Methods:

- get_ltp()
- get_market_depth()
- get_ohlc()
- get_option_chain()
- subscribe_ticks()

Implementations:

- TrueDataProvider
- TickerDataProvider
- FutureProvider

-
- Primary Provider:
 - - TrueData API + WebSocket
-
- Alternative Providers:
 - - TickerData API
 - - Exchange Approved Data Feeds
 - - Future Data Vendors
-
- Broker Integration:
 - - Kite Connect API + WebSocket
-
-
- Corporate Filings
 - - NSE India
 - - BSE India
-
- Investor Relations Sources
 - - NALCO Investor Relations Website
-
- Knowledge Sources
 - - Brokerage Website
 - - Google Drive Knowledge Repository
-
- Vector Database
 - - Qdrant
-
- Caching
 - - Redis
-
- Persistence
 - - PostgreSQL
-

Phase 3 – Backend Foundation

Build:

FastAPI

Modules:

- Authentication
- Chat
- Search
- Filings
- Stock Information

PostgreSQL

Stores:

- Users
- Sessions
- Feedback
- Analytics

Document Registry

Purpose:

Central metadata repository for all documents indexed into Qdrant.

Stores:

- Source Metadata
- Document Versions
- Citation Information
- Duplicate Detection Data
- Re-indexing Information
- Audit History

Additional PostgreSQL Table

documents

Fields:

id

source

url

company

filing_type

filing_date

document_version

checksum

created_at

updated_at

Field Definitions

id:

Unique document identifier.

source:

Document origin.

Examples:

- NSE
- BSE
- NALCO IR
- Brokerage Website
- Google Drive

url:

Original document URL.

company:

Associated company.

Example:

NALCO

filing_type:

Document category.

Examples:

- Annual Report
- Quarterly Results
- Shareholding Pattern
- Board Meeting

filing_date:

Official publication date.

document_version:

Version number of the document.

checksum:

Hash used for duplicate detection.

created_at:

First ingestion timestamp.

updated_at:

Latest processing timestamp.

Document Registry Workflow

Document Discovered

↓

Metadata Extraction

↓

Document Registry Check

↓

Duplicate Detection

↓

Version Validation

↓

Document Registration

↓

Embedding Generation

↓

Qdrant Indexing

Citation Workflow

User Query

↓

Qdrant Retrieval

↓

Document IDs Retrieved

↓

Document Registry Lookup

↓

Source Metadata Retrieval

↓

Citation Generation

↓

Response Returned

Duplicate Detection Rules

Before indexing:

1. Calculate checksum.
2. Search documents table.
3. If checksum exists:
 - Skip indexing.
4. If checksum differs:
 - Create new document version.
5. Update registry.

Redis

Stores:

- Session Cache
- Rate Limits
- Chat Context

Qdrant

Stores:

- Website embeddings
- NSE embeddings
- BSE embeddings
- NALCO embeddings

Architecture Principle

Qdrant stores vectors.

PostgreSQL stores document metadata.

Source citations must never be generated directly from vector storage.

All citation information must be retrieved from the Document Registry.

Backend Foundation Architecture Summary

FastAPI

↓

PostgreSQL

↓

Document Registry

↓

Documents Table

↓

Field Definitions

↓

Registry Workflow

↓

Citation Workflow

↓

Duplicate Detection Rules

↓

Architecture Principle

↓

Redis

↓

-
- Phase 4 – Data Ingestion Layer
 -
 - Build data ingestion before AI.
 -
 - Source 1
 -
 - Broker Website
 -
 - Example:
 - <https://yourbroker.com>
 -
 - Connection Method:
 - Sitemap Discovery + Website Crawling
 -
 - Pages to Crawl:
 -
 - - Home
 - - About Us
 - - FAQ
 - - Pricing
 - - Products
 - - KYC
 - - Open Account
 - - Algo Trading
 - - Research
 - - Contact
 -
 - Recommended Tools:
 - - Crawl4AI
 - - Firecrawl
 - - Playwright
 -
 - Flow:
 -
 - Website
 - ↓
 - Crawler
 - ↓
 - HTML Extraction
 - ↓
 - Content Cleaning
 - ↓
 - Chunking
 - ↓
 - Embeddings
 - ↓
 - Qdrant
 -
 - Refresh:
 - Daily at 2 AM
 -
 - -----
 -
 - Source 2A
 -
 - Market Data Provider Layer
 -
 - Primary Provider:
 -
 - TrueData API
 -
 - Alternative Providers:
 -
 - TickerData API
 - Exchange Approved Data Feeds
 - Future Data Vendors

-
- Connection Method:
-
- REST API
- WebSocket
-
- Purpose:
-
- - LTP
- - OHLC
- - Market Depth
- - Volume
- - Option Chain
-
- Architecture:
-
- Market Data Provider
- ↓
- REST API / WebSocket
- ↓
- FastAPI Market Service
- ↓
- Redis Cache
- ↓
- AI Bot
-
- Refresh:
-
- Real-Time
-
- Architecture Principle:
-
- Market Data must always come from APIs or WebSockets.
-
- Market Data must never be obtained through website crawling.
-
- All providers must implement the MarketDataProvider interface.

Source 2B

NSE Corporate Filings

Website:

<https://www.nseindia.com>

Connection Method:

Official APIs where available

Fallback:

Structured Crawling

Data Collected:

- Quarterly Results
- Corporate Announcements
- Shareholding Patterns
- Board Meetings
- Corporate Actions

Recommended Tools:

- Requests
- Playwright
- BeautifulSoup

Flow:

NSE

↓

Ingestion Service

↓

Document Parsing

↓

Metadata Extraction

↓

Document Registry

↓

Chunking

↓

Embeddings

↓

Qdrant

Architecture Principle:

Corporate Filings are knowledge documents.

Corporate Filings must be indexed into Qdrant.

Corporate Filings are not market data.

All filings must be stored in the Document Registry before indexing.

-
- -----
-
- Source 3
-
- BSE India
-
- Website:
- <https://www.bseindia.com>
-
- Connection Method:
-
- Structured Crawling
-
- Collect:
-
- - Results
- - Filings
- - Corporate Actions
- - Disclosures
-
- Recommended Tools:
-
- - Playwright
- - BeautifulSoup
-
- Refresh:
-
- Hourly
- Flow:
-
- BSE
- ↓
- Ingestion Service
- ↓
- Document Parsing
- ↓
- Metadata Extraction
- ↓
- Document Registry
- ↓
- Chunking
- ↓
- Embeddings
- ↓
- Qdrant
-
- Architecture Principle:

-
- Corporate Filings are knowledge documents.
-
- Corporate Filings must be indexed into Qdrant.
-
- Corporate Filings are not market data.
-
- All filings must be stored in the Document Registry before indexing.

• -----

- Source 4
-
- NALCO Investor Relations

- Website:
- <https://nalcoindia.com>

- Connection Method:

- Website Crawling

- Collect:

-
- - Annual Reports
- - Presentations
- - Financial Results
- - Press Releases

- Recommended Tools:

-
- - Crawl4AI
- - Playwright

- Refresh:

- Daily

• -----

- Source 5
-
- Google Drive Knowledge Repository

- Connection Method:

- Google Drive API

- Supported Files:

-
- - PDF
- - DOCX
- - PPTX
- - XLSX

- Purpose:

-
- - Brokerage FAQs
- - Product Documents
- - Brokerage Policies
- - Compliance Manuals
- - Research Reports
- - Internal Knowledge Documents
- - Training Material
- - Circulars

- Flow:

- Google Drive Folder
- ↓
- Google Drive API
- ↓
- Document Sync Service
- ↓
- Document Parser

- ↓
- Chunking
- ↓
- Embedding Generation
- ↓
- Qdrant
-
- Refresh:
-
- Every 15 Minutes
-

Phase 5 – Embedding Pipeline

Embedding Model

Recommended:
nomic-embed-text

Alternative:
BAAI/bge-small-en-v1.5

Requirements:

- Vector dimension must match the selected embedding model.
- Qdrant collection dimension must be generated dynamically.
- The system must never hardcode embedding dimensions.
- Fast retrieval
- Compatible with Qdrant
- Optimized for RAG

Collection:
brokerage_kb

Collection Initialization Rules

1. Load embedding model.
2. Detect vector dimension automatically.
3. Create Qdrant collection using the detected dimension.
4. Validate dimensions before vector insertion.
5. Reject mismatched vectors.

Example:

```
embedding_dimension = embedding_model.dimension
```

if collection_not_exists:

```
create_collection(
    name="brokerage_kb",
    vector_size=embedding_dimension
)
```

else:

```
validate_collection_dimension(  
    collection_name="brokerage_kb",  
    expected_dimension=embedding_dimension  
)
```

The application must not assume a fixed vector size.

Startup Initialization Requirements

During application startup:

1. Load the configured embedding model.
2. Detect embedding dimension automatically.
3. Connect to Qdrant.
4. Check whether collection "brokerage_kb" exists.
5. If collection does not exist:
 - Create collection using detected dimension.
6. If collection exists:
 - Validate dimension matches embedding model.
7. Stop startup if dimensions do not match.
8. Mark embedding service as ready.

Process

Application Startup

↓

Load Embedding Model

↓

Detect Embedding Dimension

↓

Initialize Qdrant Client



Check if Collection Exists



If Collection Does Not Exist

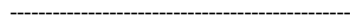
Create Collection Using Detected Dimension



Validate Collection Configuration



System Ready



Document Processing Flow

Document



Chunking



Embedding Service



Embedding Generation



Vector Validation

↓

Qdrant Storage

Metadata

- Source
- Company
- Filing Type
- Date
- Language
- Document ID
- Chunk ID
- URL
- Version
- Checksum

Collection:
brokerage_kb

- Embedding Validation
-
- At Startup:
-
- - Verify embedding model loads successfully.
- - Verify detected embedding dimension.
- - Verify Qdrant collection exists.
- - Verify collection dimension matches model.
-
- Before Indexing:
-
- - Verify vector compatibility.
- - Verify metadata integrity.
- - Verify collection availability.
-
- If mismatch detected:
-
- - Log error.
- - Stop ingestion process.
- - Notify administrator.
-
-

Reason:

Prevent corrupted vector storage and retrieval failures.

Architecture Principle

Qdrant collection creation is a startup responsibility.

Document ingestion must never create collections dynamically.

All collections must be validated before the first document is processed.

Phase 6 – RAG System

User Question



Language Detection



Intent Classification



Query Expansion



Dense Retrieval (Qdrant)



Sparse Retrieval (BM25)



Reciprocal Rank Fusion (RRF)



Metadata Filtering



Top-K Candidate Retrieval



Cross Encoder Re-Ranker



Top Relevant Chunks



Context Assembly



Ollama Cloud



Response Generation

Intent Types:

- Stock Query
- Disclosure Query
- Navigation Query
- FAQ Query
- Algo Query

Hybrid Retrieval

Purpose:

Improve retrieval quality by combining semantic search and keyword search.

Components:

Dense Retrieval:

- Qdrant Vector Search

Sparse Retrieval:

- BM25 Keyword Search

Fusion Strategy:

- Reciprocal Rank Fusion (RRF)

Benefits:

- Better retrieval of stock symbols
- Better retrieval of company names
- Better retrieval of financial terms
- Better filing search accuracy

Re-Ranking Layer

Purpose:

Improve retrieval precision before sending context to Ollama Cloud.

Problem:

Vector search retrieves semantically similar documents but may not retrieve the most relevant answer.

Solution:

Use a Cross Encoder Re-Ranker to score retrieved chunks against the user's query.

Workflow:

User Query

↓

Qdrant retrieves Top 20 chunks

↓

Cross Encoder scores all 20 chunks

↓

Select Top 5 chunks

↓

Send Top 5 chunks to Ollama Cloud

Benefits:

- Better answer accuracy
- Better citation quality
- Better handling of financial disclosures
- Reduced hallucinations
- Improved relevance ranking

Recommended Re-Ranker

Production:

- BAAI/bge-reranker-base

or

- BAAI/bge-reranker-large

Requirements:

- Query-document pair scoring
- Cross-Encoder architecture
- Compatible with financial document retrieval

Deployment:

Integrated into FastAPI Backend

Selection Strategy:

Retrieve Top 20 documents

↓

Re-rank documents

↓

Keep Top 5

↓

Send to Ollama Cloud

Claude Code Requirements

1. Retrieve Top 20 candidate chunks from Qdrant.
2. Apply metadata filtering.
3. Pass candidates to Cross Encoder Re-Ranker.
4. Score all candidate chunks.
5. Sort chunks by relevance score.
6. Select Top 5 chunks.
7. Build context window.
8. Send context to Ollama Cloud.
9. Generate response.
10. Return answer with source citations.

Implementation Rules

- Always retrieve before reranking.
- Never rerank the entire collection.
- Only rerank the Top 20 retrieved chunks.
- Always pass the Top 5 reranked chunks to Ollama Cloud.
- Preserve source metadata throughout the pipeline.
- Attach citations to every generated answer.

Architecture Principle

Vector Search optimizes for recall.

Cross Encoder Re-Ranking optimizes for precision.

The system must retrieve broadly using Qdrant and then rerank candidates before generating responses.

This retrieve-then-rerank architecture should be used for:

- NSE Filings
- BSE Filings
- Annual Reports
- Corporate Disclosures
- Investor Presentations
- NALCO Knowledge Base

Phase 7 – Multilingual Layer

Provider:

Sarvam AI

Features:

- Language Detection
- Translation
- Transliteration

Supported:

- English
- Hindi
- Tamil

Workflow:

User Language
↓
Retrieve Knowledge
↓
Generate Response
↓
Translate Response
↓
Return Response

Phase 8 – NALCO Pilot

Build the first complete company intelligence layer.

Sources:

- NSE Corporate Filings
- BSE Corporate Filings
- NALCO Investor Relations Website
- Annual Reports
- Investor Presentations
- Press Releases
- Financial Results
- Corporate Announcements
- Shareholding Patterns
- Board Meeting Outcomes
-

Questions Supported:

- Latest Results
 - Dividend History
 - Board Meetings
 - Shareholding Pattern
 - Corporate Announcements
-

Phase 9 – Website Navigation Agent

Purpose:

Guide users through the brokerage website.

Example:

User:

"How do I open a Demat account?"

Bot:

- Explains process
- Provides direct link
- Offers assistance

Data Source:

Website Sitemap
+
Website Content

Phase 10 – Production Frontend

Technology:

Next.js

Features:

- Floating Chat Widget
- Full Screen Assistant
- Citation Display
- Related Documents
- Suggested Questions
- Voice Ready Architecture

Must Support:

- Desktop
- Mobile
- Tablet

Phase 10A – Admin Dashboard

Crawler Status

NSE Sync

BSE Sync

Google Drive Sync

Indexed Documents

Embedding Count

Failed Jobs

User Feedback

Phase 11 – Scalability & Reliability

Target:

100 Concurrent Users

Requirements:

Stateless Services

Never store chat history in memory.

Use:

Redis

Background Jobs

Use workers for:

- Crawling
- Embeddings
- File Processing

Railway Deployment

Railway Services:

- Service 1:
- Next.js Frontend
-
- Service 2:
- FastAPI Backend

Re-Ranking Service

Purpose:

Re-rank retrieved chunks before LLM generation.

Deployment:

Integrated into FastAPI Backend.

Optimization:

Only rerank Top 20 retrieved chunks.

Expected Output:

Top 5 chunks passed to Ollama Cloud.

-
- Service 3:
- Qdrant
-
- Service 4:
- PostgreSQL

-
- Service 5:
- Redis
-
- Service 6:
- Background Worker
-
- Background Worker Responsibilities:
-
- - Website Crawling
- - NSE Ingestion
- - BSE Ingestion
- - NALCO Crawling
- - Google Drive Sync
- - Embedding Generation
-

Monitoring

Implement:

- Error Tracking
- Health Checks
- Logging
- Performance Metrics
- Sentry
- OpenTelemetry

Phase 12 – Algo Trading Knowledge Layer

- Research Sources:
-
- Kite Connect Documentation
- <https://kite.trade/docs/connect/v3/>
-
- NSE Retail Algo Framework
-
- NSE Circulars
-
- White Box Algorithms
-
- Black Box Algorithms
-
- Vendor Ecosystem
-
- Broker APIs
-
- Order Routing Workflows
-
- Risk Management Systems
-
- Exchange Compliance Requirements
-

Store:

- Documentation
- Circulars
- Compliance Rules

Purpose:

Educational AI assistant.

Not direct trade execution initially.

Phase 13 – Differentiators

Most competitors provide:

- FAQs
- Generic support

Provide:

Disclosure Intelligence

Explain filings in plain English.

NALCO Deep Research

Company-specific intelligence.

Multilingual Support

Hindi
Tamil
English

Source Citations

Every answer linked to source documents.

Explain Like I'm New

Convert complex filings into simple explanations.

Investor Education Mode

Explain:

- Results
- Corporate Actions
- Shareholding Changes
- Board Decisions

Phase 14 – Production Launch

Final Production Architecture

Broker Website

+

Market Data Provider Layer

(TrueData / TickerData / Future Providers)

+

NSE Corporate Filings

+

BSE Corporate Filings

+

NALCO Investor Relations

+

Google Drive Knowledge Repository

(Google Drive API)

+

NSE Algo Circulars

(Weekly Crawling)

Continuous Learning Pipeline

Broker Website Updates

↓

Automatic Crawl

↓

Qdrant Update

NSE/BSE New Filings

↓

Automatic Detection

↓

Qdrant Update

Google Drive Document Changes

↓

Automatic Sync

↓

Qdrant Update

NALCO IR Updates

↓

Automatic Crawl

↓

Qdrant Update

The AI continuously learns from brokerage website updates, regulatory disclosures, investor-relations content, and Google Drive documents without requiring any manual uploads, retraining, or developer intervention.

Phase 15

Production Data Source Registry

API Sources

- TrueData API
- TickerData API
- Kite Connect API

Market Data Sources

- TrueData
- TickerData
- Exchange Approved Feeds

Structured Filing Sources

- NSE Corporate Filings
- BSE Corporate Filings

Refresh Schedule

Market Data:
Real-Time

NSE:
Hourly

BSE:
Hourly

Broker Website:
Daily

NALCO:
Daily

Google Drive:
15 Minutes

NSE Circulars:
Weekly